Hardware/Software Design Description

For

Tennis Ball Autonomous Retrieval System

Group 17:Li Wentian, Wang Yanshan, Xiao Wenxin, Chen Guanhong

2026/5/21

Revision History

Name	Date	Reason For Changes	Version
Li Wentian	2026-03-18	Initial version.	1.0
Xiao Wenxin	2026-04-18	Import ROS and SDK.	2.0
Wang Yanshan	2026-05-01	Configure SLAM and Build a Map.	3.0
Chen Guanhong	2026-05-15	Add UI Interface.	4.0
Li Wentian	2026-05-21	Final version.	5.0

Table of Contents

1 Introduction

1.1 Purpose

This document presents a comprehensive design of the Tennis Ball Autonomous Retrieval System, an intelligent robotic system developed to address the inefficiency, time consumption, and physical effort associated with manual tennis ball collection in training environments.

The system is built upon the DJI RoboMaster EP platform and integrates an NVIDIA Jetson Orin NX edge computing module to enable real-time perception and control. By combining deep learning-based object detection, multi-threaded data processing, and precise robotic actuation, the system is capable of autonomously detecting, tracking, and collecting scattered tennis balls in dynamic environments.

Beyond basic functionality, this project emphasizes system-level design, including modular architecture, real-time performance optimization, and robustness in perception-control integration. Furthermore, the system is designed with extensibility as a core principle. Future developments will incorporate voice interaction for intuitive human-robot collaboration and obstacle avoidance capabilities using depth sensing or SLAM techniques, enabling operation in more complex and unstructured environments.

1.2 Scope

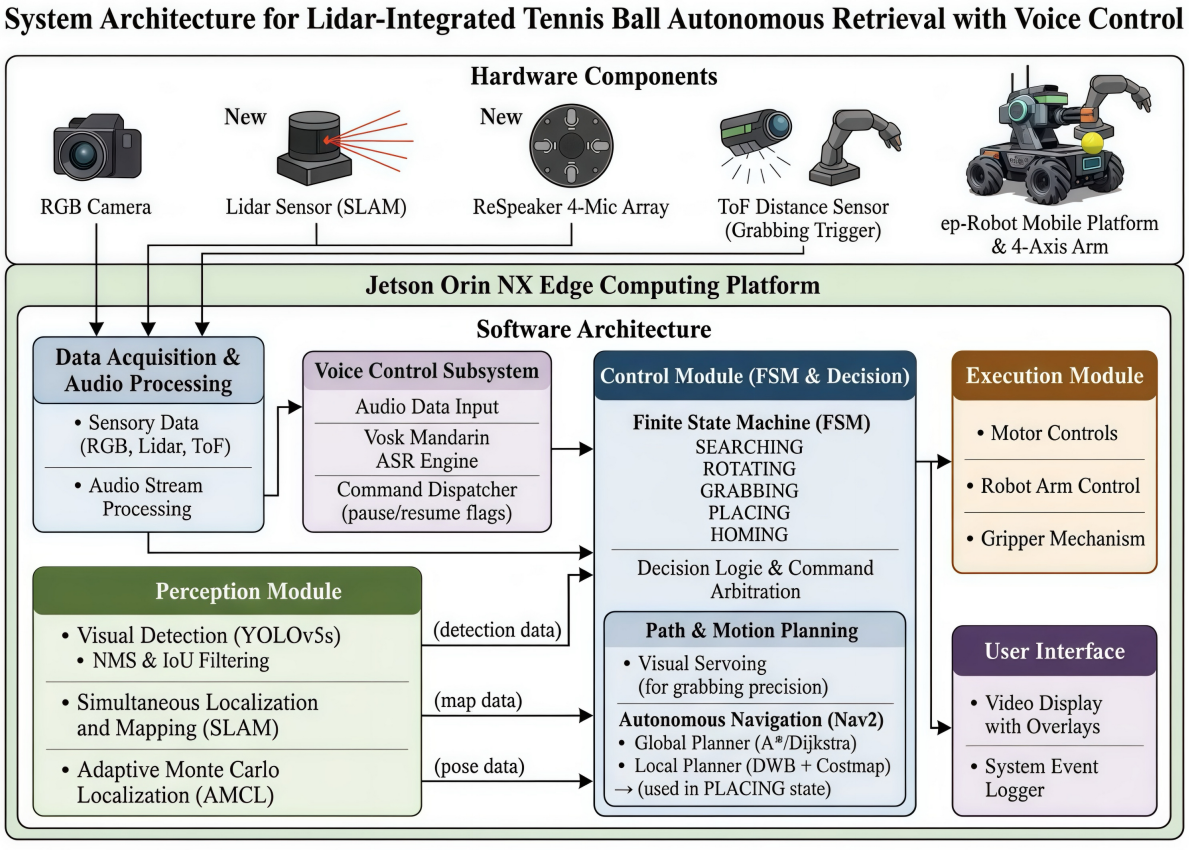

Figure 1: Framework Diagram

This document covers the end-to-end design and implementation of the Tennis Ball Autonomous Retrieval System, including both hardware and software components. Figure 1 illus-

trates the functional flowchart of the Tennis Ball Autonomous Retrieval System framework.

From a hardware perspective, the system includes camera selection and configuration, onboard computing deployment, and sensor integration strategies to support real-time perception and control.

From a software perspective, the system adopts a modular, multi-threaded architecture consisting of the following key components:

- **Data Acquisition Module:** Responsible for collecting RGB image streams, ToF distance data, and marker detection information with timestamp synchronization.
- **Perception Module:** Performs real-time tennis ball detection using a custom-trained YOLOv5s model, including preprocessing, inference, and post-processing.
- **Control Module:** Implements a finite state machine (FSM) to govern autonomous behaviors such as Searching, Rotating, Grabbing, and Placing.

Figure 2: Demonstrate the motion and grasping functions ofthe tennis robot car

- **Execution Module:** Controls the RoboMaster chassis and robotic arm to perform motion planning and grasping tasks.
- **User Interface Module:** Provides real-time visualization and logging for monitoring system status and debugging.
- **Future Extension Modules:** The system is designed to support future enhancements, including:
  - Voice control for human-robot interaction.
  - Obstacle avoidance and navigation using RGB-D sensors or SLAM-based methods.

1.3 Definitions, Acronyms and Abbreviations

The following terms and abbreviations are used throughout this document:

- **RoboMaster EP:** A programmable mobile robot platform developed by DJI.
- **YOLOv5s:** A lightweight object detection model optimized for real-time inference.
- **Jetson Orin NX:** An embedded AI computing platform for edge processing.

- **FSM (Finite State Machine):** A control model managing system behavior via states and transitions.
- **ToF (Time-of-Flight):** A sensor measuring distance using light propagation time.
- **RGB-D Camera:** A sensor providing both RGB images and depth information.
- **NMS (Non-Maximum Suppression):** A method to remove redundant detection boxes.
- **IoU (Intersection over Union):** A metric measuring overlap between bounding boxes.
- **SLAM (Simultaneous Localization and Mapping):** A technique for building maps while tracking position.

1.4 References

The development of the Tennis Ball Autonomous Retrieval System relies on a combination of official hardware specifications, software development frameworks, and open-source datasets. Technical standards and communication protocols are derived from the official DJI RoboMaster EP documentation and user manuals. The perception system is built upon the YOLOv5 architecture by Ultralytics, utilizing specialized tennis ball datasets hosted on Roboflow for model training and optimization. Furthermore, navigation baselines are informed by the ICRA RoboMaster Sim2Real challenge resources. A complete list of all bibliographic references cited in this document is provided in the **References** section at the end of this report.

1.5 Organization of Document

This document is organized as follows:

- Hardware design
- Camera Selection
- Camera Setup
- Basic Computer Requirement
- Software Modules Design
- Architecture Overview
- Video I/O Package
- Pre-Preprocessing Package
- Video Processing Package
- Event Recognition Package
- Presentation Package
- Class Design
- User Interface Design

2 Hardware Design

2.1 Camera Selection

The design of the vision system for the RoboMaster EP encompasses two parts: **Utilization of the native camera** and **Selection of additional/expansion cameras**.

2.1.1 Analysis of the Native Camera

The RoboMaster EP is equipped with a high-performance camera integrated into the gimbal. This is the preferred choice for initial project development and basic vision tasks. Its core specifications are as follows [3]:

- **Image Sensor**: 1/4-inch CMOS, 5 effective megapixels.
- **Lens Field of View (FOV)**: 120° (wide-angle), suitable for close-range, wide-area observation.
- **Resolution**:
  - **Photo**: Up to 2560×1440.
  - **Video Recording**: FHD (1920×1080 @ 30fps), HD (1280×720 @ 30fps).
- **Video Transmission Quality**: Real-time transmission quality is 720p@30fps.
- **Storage Format**: Image: JPEG; Video: MP4 (Max bitrate: 16 Mbps).
- **Storage Media**: Supports microSD cards up to 64GB.

Evaluation: This camera is suitable for tasks such as visual line following, ArUco marker recognition, and color tracking in low-to-medium speed scenarios. However, its 30fps frame rate may present limitations for detecting high-speed moving objects.

2.1.2 Recommendations for External/Expansion Cameras

For tasks requiring high-precision environmental perception such as Simultaneous Localization and Mapping (SLAM), 3D reconstruction, or deep learning-based object detection, relying solely on the native monocular camera is insufficient. The RoboMaster EP supports the integration of third-party hardware via the sensor adapter module (IO/AD interface) or USB ports on the intelligent controller [1]. It is recommended to select expansion cameras based on the following scenarios:

Application Scenario	Recommended Camera Type	Selection Rationale & Key Parameter Suggestions
Obstacle Avoidance & SLAM	**Depth Camera / RGB-D Camera**	Directly acquires depth information of the environment for mapping and obstacle avoidance. Recommended models: Intel RealSense D435 or D455 [**realsense˙specs**].
AI Vision	**High-Definition USB Camera**	Used in conjunction with edge computing devices for deep learning inference. Requires support for frame rates above 60fps to reduce motion blur. Lenses with low distortion are preferred.
Research & Algorithm Validation	**LiDAR + Monocular Vision**	Combines point cloud data from LiDAR with texture information from the camera for advanced perception. Commonly seen in research scenarios like the ICRA RoboMaster Challenge [5].

Table 2: External Camera Selection Guide

2.1.3 Selected Camera for the Tennis-Bot System

After evaluating the native camera's limitations (30fps, high latency), the project adopts a **Logitech C270 USB camera** (or equivalent) connected directly to the Jetson Orin NX. This camera provides:

- 640×360 resolution at 30fps (configurable up to 1280×720)
- UVC compliant, natively supported by ROS 2 `usb_cam` driver
- Low latency (50ms) and stable frame timing

The camera is mounted on the EP's gimbal and publishes RGB images on the topic `/camera/image_color`, which is subscribed by the `TennisPickNode` for YOLOv5 detection. This configuration ensures consistent frame delivery even under heavy CPU/GPU load.

2.2 Camera Setup

2.2.1 Native Camera Settings

Key configurations for the native camera can be made via the RoboMaster App or SDK:

- **Exposure & White Balance**: Adjust according to the experimental environment lighting. In indoor lighting conditions, it is recommended to set fixed values (manual mode) to

prevent image flickering or color shifts caused by automatic adjustments. If the ambient light is sufficient, prioritize increasing the shutter speed to reduce motion blur when the robot is moving.

- **Video Recording Resolution**: If the task requires recording high-definition video for offline analysis, set it to `FHD (1920x1080)`. If it is only for real-time algorithm processing, it is advisable to lower the resolution to improve transmission speed and reduce latency.
- **Video Transmission Bitrate**: The maximum real-time video transmission bitrate is 6 Mbps [3]. In poor network conditions, the bitrate can be appropriately reduced to ensure smooth footage.

2.2.2 Expansion Camera Settings (Example: RGB-D Camera)

- **Synchronized Triggering**: If using multiple sensors simultaneously, hardware triggering or software timestamp synchronization must be configured to ensure the precision of data fusion.
- **Resolution & Frame Rate Balance**: The higher the resolution of the depth camera, the larger the data volume, and the higher the demand on computer performance. It is recommended to choose a moderate resolution (e.g., 640x480 @ 30fps) that meets accuracy requirements to ensure system real-time performance.

2.3 Basic Computer Configuration Requirements

The intelligent controller of the RoboMaster EP is primarily responsible for video transmission and chassis control. Complex vision processing algorithms (e.g., deep learning, V-SLAM) require an additional **onboard computer** or a **host computer** to handle the computation. The computer can communicate with the robot by connecting to the EP's Wi-Fi network (Router Mode) or via the SDK [1].

2.3.1 Basic Development Configuration (Educational Programming)

If the project primarily uses DJI's official graphical programming environment or simple Python APIs (without involving complex AI models), a standard personal computer connected to the robot's Wi-Fi is sufficient. Based on the reference configuration from the DJI Education Platform [2]:

- **Operating System**: Windows 10 64-bit or Windows 7 64-bit.
- **Processor (CPU)**: Intel Core i5-7200U (7th Gen) or higher.
- **Memory (RAM)**: 4GB or more.
- **Graphics Card (GPU)**: Integrated graphics (e.g., Intel HD Graphics 620) is acceptable; a dedicated GPU is preferred.

2.3.2 Advanced Development Configuration (Onboard Edge Computing)

To achieve autonomous navigation and AI recognition, a microcomputer needs to be mounted on the robot itself (e.g., NVIDIA Jetson series). Referring to the hardware configuration used in challenges like the ICRA RoboMaster Sim2Real Challenge [5], a standard onboard computer setup includes:

- **Core Requirement**: The CPU needs multi-core processing capability to handle real-time data transmission and algorithm computation.
- **Recommended Configuration**: Intel NUC11PAHI7 (or equivalent performance device).
  - **CPU**: Intel Core i7-1165G7 (2.8GHz, 8 threads).
  - **Memory**: 8GB DDR4.
  - **Storage**: 256GB NVMe SSD (for system and data caching).
- **Expansion Interfaces**: Must have USB 3.0 ports for connecting RGB-D cameras. It should also support the 5.8GHz Wi-Fi band to ensure low-latency communication with the EP robot [4].

2.3.3 Power Supply and Integration Considerations

- **Power Scheme**: The onboard computer requires an independent power source. The RoboMaster EP's power management module provides a TX30 power port (12V/5A output) and a USB Type-A port (5V/2A) [3], which can be used to power some smaller computing devices or routers. If adding a high-power computer (e.g., NUC), it is recommended to use a separate LiPo battery to avoid affecting the robot's chassis motor runtime.
- **Physical Mounting**: The EP chassis and base plate have pre-drilled holes with 8mm and 16mm spacing, facilitating the installation of expansion plates to securely mount the computer.

2.3.4 Onboard Edge Computer: NVIDIA Jetson Orin NX

The actual system is deployed on an **NVIDIA Jetson Orin NX** (16GB or 8GB module) running Ubuntu 22.04 and ROS 2 Humble. This module provides:

- 6-core Arm Cortex-A78AE CPU (2.2GHz)
- 1024-core Ampere GPU with 32 Tensor Cores (up to 100 TOPS)
- 8/16GB LPDDR5 RAM
- Built-in Wi-Fi 5 and Gigabit Ethernet for communication with the RoboMaster EP

The Jetson handles YOLOv5 inference, Nav2 planning, voice recognition, and all ROS 2 nodes simultaneously. Power is supplied via a 5V/4A DC-DC converter connected to the EP's TX30 power port (12V/5A) or an independent Li-Po battery.

2.4 ReSpeaker Microphone Array

To enable future voice interaction capabilities (as outlined in Section 1), the system integrates a ReSpeaker 4-Mic Array based on the XVF-3000 voice processor. This microphone array provides high-quality far-field voice capture, keyword spotting, and direction-of-arrival (DoA) estimation, making it ideal for human-robot voice command interfaces in noisy environments.

2.4.1 Specifications and Features

The ReSpeaker microphone array used in this project incorporates the following key specifications (based on the XVF-3000 core and MP34DT01-M MEMS microphones):

Parameter	**Description**
Processor	XVF-3000: 16 real-time logic cores on two xCore tiles, dual-issue mode up to 2400 MIPS
Memory	512 KB internal single-cycle SRAM, 2 MB built-in flash, 16 KB OTP (max 8 KB per tile)
Microphone Array	4× MP34DT01-M digital PDM microphones
Microphone Sensitivity	–26 dB FS (full-scale)
SNR	61 dB
Acoustic Overload Point	120 dB SPL
LED Indicators	12× APA102 RGB LEDs with 256-level brightness, 800 kHz data transmission
Audio Output	Wolfson WM8960 codec, 3.5 mm aux jack, 16-bit/24-bit stereo at 16 kHz, 40 mW into 16 @ 3.3V
Power Supply	5V via Micro USB or expansion header; typical power consumption 190 mA
USB Compliance	Full USB 2.0 with integrated PHY, supports DFU mode

Table 3: ReSpeaker microphone array key specifications

2.4.2 Integration with the Retrieval System

The ReSpeaker array is connected to the NVIDIA Jetson Orin NX edge computer via USB (USB 2.0 interface). The device is recognized as a standard USB audio card, enabling easy integration with speech recognition frameworks such as Vosk, Whisper, or custom keyword spotting engines. The built-in XVF-3000 performs acoustic echo cancellation (AEC), beamforming, and DoA estimation, reducing the processing load on the Jetson.

The software architecture (Section 3) will be extended with a dedicated `VoiceController` class (see Section 3.3.5) that:

- Captures audio from the ReSpeaker array in real time.
- Performs keyword detection (e.g., “stop”, “go”, “search”) using a lightweight on-device model.
- Optionally estimates the direction of the speaker using DoA data, allowing the robot to rotate towards the user for better interaction.

- Sends voice commands to the finite state machine (Section 3.1.4) to override or augment autonomous behavior.

The microphone array is physically mounted on the robot chassis facing forward, above the camera gimbal, to minimize mechanical noise from the motors. Power is supplied through the RoboMaster EP's USB port (5V/2A) or an independent 5V regulator, depending on overall system power budgeting.

2.4.3 Future Voice Interaction Scenarios

With the ReSpeaker array, the Tennis Ball Autonomous Retrieval System will support:

- Hands-free operation: starting/stopping collection, resetting the state machine.
- Multi-user command filtering: using DoA to focus on the nearest speaker.

This hardware selection aligns with the project's extensibility goal (Section 1.1) and provides a low-latency, robust foundation for human-robot collaboration.

2.5 YDLIDAR X3 Pro LiDAR

To enable robust obstacle avoidance and autonomous navigation in complex environments (as outlined in Section 1), the system incorporates a YDLIDAR X3 Pro LiDAR sensor. This 2D LiDAR provides high-frequency environmental scanning, delivering reliable distance measurements for real-time mapping and collision avoidance. It complements the vision-based perception system, especially under low-light conditions or when depth information from monocular cameras is insufficient.

2.5.1 Specifications and Features

The YDLIDAR X3 Pro is a triangulation-based 2D LiDAR offering a balanced combination of measurement range, sampling rate, and compact form factor. Table 4 lists its key parameters.

Parameter	**Description**
Brand / Model	EAI YDLIDAR X3 Pro
Measuring Principle	Triangulation
Measurement Radius	8m @ 80% reflectivity (typical)
Minimum Measurement Distance	0.12m
Distance Accuracy	5% @ 8m
Sampling Rate	4000 samples per second
Scan Frequency	5Hz – 10Hz (adjustable)
Ambient Light Immunity	Up to 40Klux (suitable for outdoor use)
Angular Resolution	0.6° – 1.2° (adjustable)
Communication Speed	115.2kbps (UART)
Power Supply / Consumption	5V / 1.75W
Dimensions (mm)	$95 \times 68.9 \times 40.5$
Weight	135g

Table 4: YDLIDAR X3 Pro LiDAR specifications

2.5.2 Integration with the Retrieval System

The YDLIDAR X3 Pro is connected to the NVIDIA Jetson Orin NX via a USB-to-UART adapter (the sensor uses TTL serial communication). The driver provided by YDLIDAR (e.g., `ydlidar_ros2` or the SDK) is used to parse the laser scan data at up to 4000 points per second. The sensor is mounted on the front of the robot chassis, approximately 10cm above ground level, to ensure a clear view of obstacles such as tennis ball collection boxes, walls, or human legs. Power is supplied from the RoboMaster EP's 5V expansion port or a dedicated 5V regulator, with a measured current draw of 350mA.

In the software architecture (Section 3), a new `LidarHandler` class will be introduced, running in a dedicated thread. Its responsibilities include:

- Collecting raw laser scan data and converting it to Cartesian coordinates.
- Publishing a simplified obstacle list (e.g., the closest point in each sector) to a thread-safe queue.
- Providing emergency stop signals when an obstacle enters a predefined safety zone (e.g., within 30cm).

The finite state machine (Section 3.1.4) will be extended to incorporate obstacle avoidance behaviour. When the robot is in `SEARCHING` or `GRABBING` states and the LiDAR detects an obstacle in the forward path, the robot will either stop and wait, or execute a small evasive rotation (e.g., turn 20° away) before resuming normal operation.

2.5.3 Applications in Future Navigation and SLAM

The YDLIDAR X3 Pro enables advanced capabilities beyond basic obstacle avoidance:

- **SLAM (Simultaneous Localization and Mapping):** Using packages such as `Gmapping`, `Hector SLAM`, or `CartoGrapher`, the LiDAR data can be fused with wheel odometry to create a 2D occupancy grid map of the tennis court. This allows the robot to localise itself and plan global paths for systematic ball collection.
- **Autonomous Exploration:** Combined with the vision module, the robot can navigate to unexplored areas, ensuring full coverage of the playing field.
- **People Avoidance:** The LiDAR's 8m range and immunity to ambient light make it possible to detect moving people (e.g., players or coaches) and gracefully avoid collisions, enhancing safety.

By integrating the YDLIDAR X3 Pro, the Tennis Ball Autonomous Retrieval System gains a robust distance sensing modality that is independent of lighting conditions, complementing the RGB camera and ToF sensor already present. This hardware addition directly supports the project's long-term goal of autonomous operation in unstructured, dynamic environments (see Section 1.1).

2.6 Time-of-Flight Sensors

The system uses two complementary Time-of-Flight distance sensors to enable reliable grasping: the built-in ToF module of the RoboMaster EP and an optional external Y-TOF10M module for higher precision.

2.6.1 EP Built-in ToF Sensor

The RoboMaster EP is equipped with a forward-facing infrared ToF sensor that publishes data on the ROS topic `/range_0`. Key specifications:

- **Range:** 0.1m – 8.0m
- **Field of View:** 15°
- **Update rate:** up to 100Hz (via SDK callback)
- **Trigger threshold in grasping:** 0.13m (as configured in the state machine)

This sensor provides the primary stop signal during the `GRABBING` state. Its wider FoV ensures that the ball is detected even if the robot approaches slightly off-centre.

2.6.2 Optional Y-TOF10M Module (Future Enhancement)

For applications requiring higher accuracy or a narrower beam (2°) to discriminate the ball from nearby obstacles, the system supports the Y-TOF10M module described in Table **??**. It can be mounted on the gripper and interfaced via UART or I²C. However, the current implementation relies exclusively on the EP's built-in ToF to minimise wiring and software complexity.

2.7 Robotic Arm and Gripper

The DJI RoboMaster EP is equipped with a 2-degree-of-freedom robotic arm and a parallel gripper. These actuators are controlled via ROS actions (`/move_arm`, `/gripper`) provided by the `robomaster_ros` package.

2.7.1 Arm Specifications

- **Degrees of freedom:** 2 (X-Z planar motion relative to chassis)
- **Workspace:** X range 0.05m – 0.22m, Z range –0.12m to 0.15m (relative to arm base)
- **Motion accuracy:** ±0.005m (open-loop control)
- **Max payload:** 0.5kg (including gripper and tennis ball)

Two key poses are predefined in the software:

- **Home pose:** `(x=0.18m, z={0.07m)` – stowed position for searching and navigation.
- **Lift pose:** `(x=0.08m, z=0.11m)` – raised after grasping to transport the ball.

2.7.2 Gripper Specifications

- **Type:** Parallel jaw, servo-driven
- **Max opening width:** 0.06m (sufficient to enclose a tennis ball, diameter 0.067m)
- **Control modes:** Open, close, pause (with configurable power from 0.0 to 1.0)
- **Grasp detection:** Current feedback; the gripper closes until a current surge indicates the ball is secured

The gripper is controlled via the `GripperControl` action, with power set to 0.8 during normal operation.

2.8 Power Supply and Mechanical Integration

2.8.1 Power Budget and Distribution

The total power consumption of all onboard components is summarised in Table 5.

Component	Voltage	Typical Power
Jetson Orin NX (15W mode)	5V (via converter)	15W
YDLidar X3 Pro	5V	1.75W
ReSpeaker 4-Mic Array	5V	0.95W (190mA)
RoboMaster EP (chassis + arm)	12V (internal)	— (powered separately)

Table 5: Power budget of external modules

The EP's intelligent battery provides 12V through a TX30 power port (rated 5A continuous). A 12V-to-5V DC-DC converter (e.g., LM2596) is used to power the Jetson, YDLidar, and ReSpeaker. Alternatively, a dedicated 5V/4A USB-C power bank can be mounted on the chassis for testing.

2.8.2 Mechanical Mounting

All sensors and the edge computer are attached to the EP's expansion plates using the pre-drilled 8mm and 16mm mounting holes. Figure **??** shows the final assembly:

- The **Jetson Orin NX** is housed in a 3D-printed enclosure mounted on the rear of the chassis, above the battery.
- The **YDLidar X3 Pro** is placed at the front, 10cm above ground, aligned with the robot's forward axis.
- The **ReSpeaker 4-Mic array** is attached to the top of the gimbal mount, facing forward and upward to minimise motor noise.
- The **USB camera** (e.g., Logitech C270) is mounted on the gimbal, replacing or augmenting the EP's native camera. Its optical axis is aligned with the gripper's centre.

All USB cables are secured with cable clips to prevent interference with the arm motion.

3 Software Modules Design

3.1 Architecture Overview

The Tennis-Bot system is built upon ROS 2 Humble and follows a modular, layered architecture that integrates perception, planning, control, and actuation into a cohesive state-machine-driven pipeline. At the Input Layer, the robot collects multi-modal sensory data through four primary devices: a USB Camera streaming raw images, a YDLidar X3 providing 2D laser scans, a ToF IR Sensor publishing distance measurements, and a ReSpeaker 4-Mic Array capturing raw audio for voice interaction. These heterogeneous inputs feed into the Perception Planning Layer, where specialized compute nodes process and fuse the data: a YOLOv5 Detector performs real-time tennis ball detection and outputs bounding boxes; SLAM Toolbox with AMCL handles simultaneous localization and mapping, producing both the robot's pose and an occupancy map; an EKF fuses wheel odometry, IMU data, and ToF readings into a filtered odometry estimate; and a Vosk Chinese ASR node runs in a separate multiprocessing process to enable voice command recognition, with pause/resume control gated by a dedicated logic switch.

The Control State Machine Layer serves as the system's behavioral core, implementing a deterministic finite-state machine with five sequential states: SEARCHING $\rightarrow$ ROTATING $\rightarrow$ GRABBING $\rightarrow$ PLACING $\rightarrow$ HOMING. State transitions are triggered by perceptual thresholds—for instance, the system transitions from SEARCHING to ROTATING when the YOLOv5 detector registers at least 3 consecutive frames with confidence above 0.7, and from GRABBING to PLACING when the ToF sensor reports a distance 0.13 m with the gripper closed. The HOMING state leverages Nav2's NavigateToPose action to return the robot to its origin, after which the cycle can resume. Finally, the Actuation Layer translates high-level state commands into physical motion through four subsystems: a Mecanum Chassis executing velocity commands, a Robotic Arm controlled via MoveArm actions, a Gripper operated through GripperControl actions, and the Nav2 Stack providing global path planning (A*) and local trajectory generation over a dynamic costmap. This layered design ensures clear separation of concerns, with dashed feedback lines reserved for asynchronous sensor updates and closed-loop control, enabling robust autonomous tennis ball collection in a structured indoor environment.

3.1.1 Video I/O Package

This package is responsible for acquiring raw data from the robot's sensors. It includes three subcomponents:

- **Camera stream:** A dedicated thread captures RGB frames from the RoboMaster EP camera at 30fps using the RoboMaster SDK. The frames are pushed into a thread-safe queue (`frame_queue`) for further processing.
- **ToF sensor:** The Time-of-Flight distance data is subscribed at 10Hz via the SDK's callback mechanism. The latest distance value is stored in a shared variable protected by a lock (or updated atomically).
- **Marker detection:** The SDK's built-in ArUco marker detector is subscribed. For each detected marker, a callback provides its normalized coordinates, size, and ID. The marker information is stored in a list that is updated in real time.

All sensor data is timestamped to enable synchronization. The video capture thread runs independently and ensures that the frame queue never overflows by discarding the oldest frame when the queue is full.

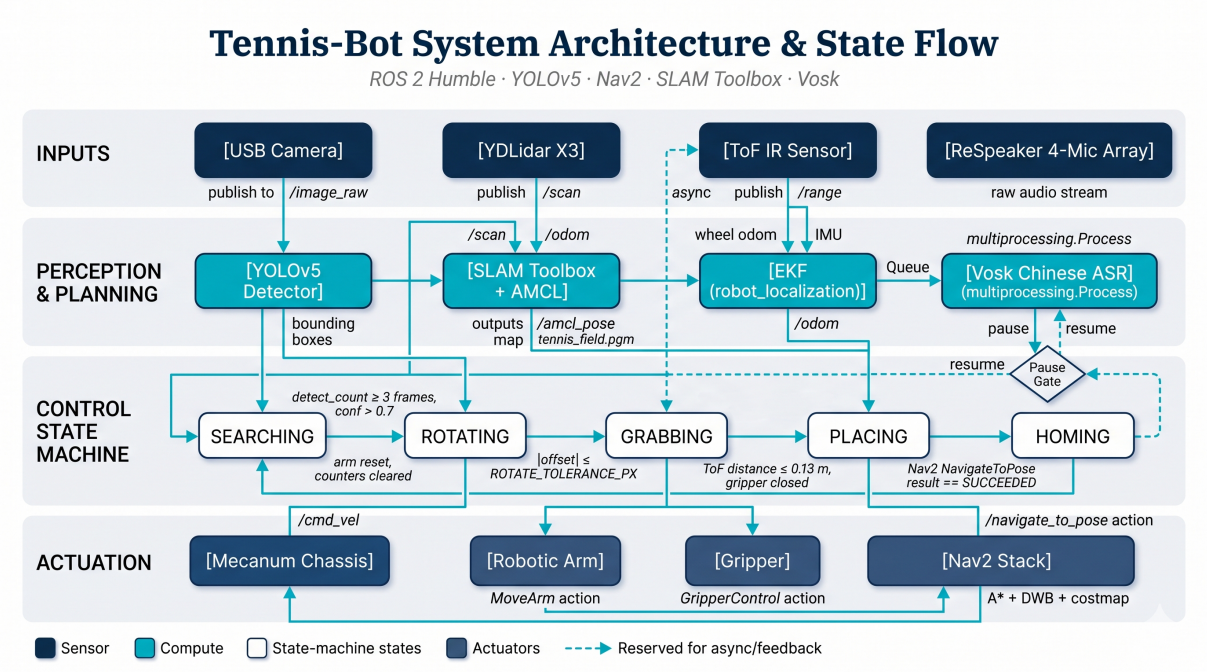

Figure 3: Software architecture overview

3.1.2 Pre-Preprocessing Package

The raw RGB frame obtained from the Video I/O package must be transformed into a format suitable for the neural network. The preprocessing steps are performed inside the detection thread and include:

- **Letterbox resizing:** The image is resized to 640×640 while preserving the original aspect ratio. Padding is added to the borders to achieve the square size required by YOLOv5[6].
- **Color space conversion:** The image is converted from BGR (OpenCV default) to RGB, and pixel values are normalized to the range $[0, 1]$.
- **Tensor conversion:** The resulting array is converted to a PyTorch tensor, moved to the GPU (if available), and half-precision (FP16) is used to accelerate inference.

This package is implemented in the `preprocess()` function and is called immediately before inference.

3.1.3 Video Processing Package

The core computer vision tasks are carried out in this package. It runs in a separate thread and performs the following operations:

- **YOLOv5s inference:** The preprocessed tensor is fed into a YOLOv5s model that has been retrained on a custom tennis ball dataset. The model runs on the Jetson's GPU, achieving inference times below 35ms per frame.
- **Post-processing:** The raw output is filtered using non-maximum suppression (NMS) with an IoU threshold of 0.45. Only detections with a confidence score above 0.7 are retained.

- **Target selection:** Among all tennis ball detections, the one with the largest bounding box area is selected as the current target. This heuristic corresponds to the physically closest ball.
- **Marker integration:** The marker list (from the Video I/O package) is also available here; it is used later during the placing state to estimate the distance to the collection box.

The detection results (bounding boxes, confidences, class names, and the selected target) are packed into a dictionary and pushed into `detection_queue` for use by the state machine and the presentation package.

3.1.4 Event Recognition Package

This package implements the finite state machine that governs the robot's autonomous behavior. It runs in the control thread and continuously reads the latest detection results from `detection_queue`. The state machine consists of four states, as originally designed in the project:

SEARCHING: When no tennis ball is detected for five consecutive frames, the robot performs a systematic search. It rotates the chassis in 15° steps at 60°/s while keeping the robotic arm lifted to avoid blind spots. The search direction alternates to cover 360°.

ROTATING: Once a ball is detected with high confidence (> 0.7), the robot enters this state. It calculates the horizontal offset between the ball's center and the image center (640pixels). A proportional controller computes a rotation angle: angle $= (\text{offset}/600) \times 50°$, clamped to ±15°. The robot rotates until the offset is less than 30pixels, at which point it transitions to GRABBING.

GRABBING: After alignment, the robot approaches the ball at a low speed (0.2m/s) while monitoring the ToF distance. When the distance falls between 13mm , it stops and closes the gripper. A current surge in the gripper motor indicates successful grasping. If the distance does not reach the threshold within 10seconds, the robot aborts and returns to SEARCHING.

PLACING: The robot searches for the collection bin using the pre-built SLAM map and the Nav2 navigation stack. The bin's position (in the `map` frame) is either loaded from ROS parameters (`box_x`, `box_y`, `box_yaw_deg`) or recorded as the robot's starting pose (the "home" position). A `NavigateToPose` action goal is sent to the Nav2 action server. The server uses A* global planning and the DWB local controller to avoid obstacles while driving toward the bin. During navigation, the state machine monitors the goal status; if the goal is aborted or cancelled (e.g., by voice pause), the robot will retry up to two times. Upon successful arrival (status `SUCCEEDED`), the gripper opens to release the ball, then the robot reverses 0.5m and rotates 180° to exit the bin area.

HOMING: The robotic arm returns to its home position (`ARM_HOME_X`, `ARM_HOME_Z`) and the gripper opens. Detection counters are cleared. The state machine then loops back to SEARCHING.

State transitions are triggered by events (e.g., detection loss, successful alignment, grasp completion) and are handled in a thread-safe manner. The package calls the appropriate robot control functions via the RobotController class.

3.1.5 Presentation Package

This package provides real-time visualization for debugging and monitoring. It runs in the main thread and performs the following tasks:

- **Overlay drawing:** For each detected tennis ball, a green bounding box is drawn with the class name and confidence. The current robot state is displayed in the top-left corner. Detected markers are outlined in white with their ID.
- **Video display:** The annotated frame is shown in an OpenCV window titled "RoboMaster Loop Grab". The window refreshes at the camera frame rate.
- **User interaction:** The program can be terminated by pressing the "ESC" key.

The presentation package retrieves the latest frame and detection results from `detection_queue` (with a timeout to avoid blocking). It does not perform any processing that could delay the control loop.

3.1.6 Navigation Initialization

At system startup, the node performs the following steps to ensure reliable autonomous navigation:

1. Waits for the `/navigate_to_pose` action server (Nav2) to become available.
2. Attempts to read the current robot pose from TF (`map` → `base_link`).
3. If no valid TF exists, but the user has provided bin coordinates via ROS parameters, the node publishes those coordinates as an initial pose to the `/initialpose` topic to seed the AMCL particle filter.
4. Otherwise, it logs a warning and expects the operator to manually set the initial pose in RViz.

This procedure dramatically reduces the time required for AMCL convergence and allows fully autonomous starts.

3.2 Voice Control Module

To enable hands-free operation and emergency override, the system integrates a voice command interface using the ReSpeaker 4-Mic array and the Vosk offline speech recognition engine. Voice processing runs in a dedicated **multiprocessing** subprocess to avoid blocking the main ROS 2 threads due to Python's GIL.

3.2.1 Architecture

- A separate Python process (started via `multiprocessing.get_context('spawn')`) continuously captures audio from the USB microphone at 16kHz mono, applies software gain (8×) to compensate for low input levels, and feeds the PCM stream to a Vosk recognizer.
- The recognizer is configured with a small grammar containing the keywords "stop", "begin", "continue", "go". Partial and final results are evaluated in real time.

- When a keyword is matched, the subprocess pushes a string (`'pause'` or `'resume'`) into a thread-safe `multiprocessing.Queue`.
- The main process runs a lightweight dispatcher thread that reads from this queue and invokes the registered callbacks (`on_pause()`, `on_resume()`) in the main thread context.

3.2.2 Integration with State Machine

The voice callbacks perform the following actions:

- `on_pause()`: Sets an internal `Event` flag, publishes a zero-velocity `Twist` to stop the chassis, and cancels any active Nav2 navigation goal.
- `on_resume()`: Clears the pause flag, allowing the control loop to continue. The state machine will automatically re-attempt the interrupted action (e.g., re-sending the Nav2 goal) when it exits the waiting loop.

All motion primitives (e.g., `chassis_move`, `arm_move`) block until the pause flag is cleared, ensuring the robot never moves while voice-paused.

3.3 Class Design

The software is decomposed into several key classes, each encapsulating a distinct responsibility. The relationships among them follow the architecture described above.

3.3.1 CameraHandler Class

Class Name: CameraHandler

Inherited class: None

Aggregated Classes: None

Associated Classes: BallDetector (provides frames), StateMachine (provides marker and ToF data)

Data Members:

Type	Name
`robot.Robot`	`ep_robot`
`camera.Camera`	`ep_camera`
`sensor.Sensor`	`ep_tof`
`vision.Vision`	`ep_vision`
`queue.Queue`	`frame_queue`
`float`	`_tof_dist` (atomic)
`list`	`markers`
`bool`	`running`

Member Functions:

Return Type	Name
void	_tof_cb(data)
void	_marker_cb(marker_info)
void	video_thread()
cv::Mat	get_frame() (non-blocking)
float	get_tof()
list	get_markers()

Data Structures:

- frame_queue: a thread-safe queue (FIFO) holding the most recent camera frames.
- markers: a Python list of MarkerInfo objects (custom class) containing normalized coordinates, size, and ID.

Processing:

The CameraHandler initializes the robot connection and starts the video stream. It subscribes to ToF and marker detection callbacks. A dedicated thread (video_thread) continuously reads frames from the camera and pushes them into frame_queue. The ToF callback updates a shared variable; the marker callback updates the marker list. All data is made available to other classes through getter methods.

3.3.2 BallDetector Class

Class Name: BallDetector

Inherited class: None

Aggregated Classes: YOLOv5 model (via DetectMultiBackend)

Associated Classes: CameraHandler (receives frames), StateMachine (outputs detections)

Data Members:

Type	Name
torch.device	device
DetectMultiBackend	model
float	stride
list	names (class names)
queue.Queue	detection_queue
bool	running

Member Functions:

Return Type	Name
numpy.ndarray	preprocess(frame)
list	detect(frame)
void	detection_thread()
dict	_select_nearest_ball(detections)

Data Structures:

- `detection_queue`: a queue that holds tuples of `(frame, detections_list)`.
- Detections are stored as a list of dictionaries, each containing keys: `'bbox'`,`'confidence'`, `'class'`,`'class_name'`.

Processing:

The BallDetector runs a separate thread that repeatedly:

1. Retrieves a frame from `frame_queue` (provided by CameraHandler).
2. Preprocesses the frame (letterbox, normalization, tensor conversion).
3. Performs YOLOv5 inference.
4. Applies NMS and filters low-confidence detections.
5. Selects the nearest ball (largest bounding box area) if any tennis ball is found.
6. Packs the results into a dictionary and puts them into `detection_queue`.

The thread runs as long as `running` is true.

3.3.3 RobotController Class

Class Name: RobotController

Inherited class: None

Aggregated Classes: RoboMaster robot modules (`chassis`, `arm`, `gripper`)

Associated Classes: StateMachine (receives commands)

Data Members:

Type	Name
`robomaster.robot.Chassis`	`chassis`
`robomaster.robot.RoboticArm`	`arm`
`robomaster.robot.Gripper`	`gripper`

Member Functions:

Return Type	Name
`void`	`move(x, y, z, z_speed)`
`void`	`drive_speed(vx, vy, vz)`
`void`	`stop()`
`void`	`rotate(angle)`
`void`	`open_gripper(power)`
`void`	`close_gripper(power)`
`void`	`arm_moveto(x, y)`
`void`	`arm_recenter()`

Data Structures:

None (directly uses the RoboMaster SDK functions).

Processing:

The RobotController provides a high-level abstraction over the RoboMaster SDK. All movement functions are blocking (they wait for completion) except `drive_speed` which sets a continuous speed. It is used by the StateMachine to execute state-specific actions such as rotating, moving forward, and controlling the arm/gripper.

3.3.4 StateMachine Class

Class Name: StateMachine

Inherited class: None

Aggregated Classes: BallDetector, RobotController

Associated Classes: CameraHandler (indirectly via BallDetector)

Data Members:

Type	Name
`str`	`current_state`
`list`	`target_bbox`
`int`	`search_direction` (±1)
`float`	`search_angle` (15°)
`bool`	`running`

Member Functions:

Return Type	Name
`void`	`control_thread()`
`void`	`_small_angle_search()`
`bool`	`_rotate_to_center(target_bbox)`
`bool`	`_grab_once()`
`bool`	`_place_ball()`
`void`	`_clear_queues()`

Data Structures:

- The detection results are read from `detection_queue` (owned by BallDetector).
- ToF distance and marker list are accessed via CameraHandler's getters.

Processing:

The StateMachine runs the control thread, which implements the four-state behavior described in Section 3.1.4. It continuously polls `detection_queue` for new results, evaluates the current state, and invokes appropriate RobotController methods. State transitions are guarded by conditions and timeouts to ensure robustness. After completing a PLACING cycle, the queues are cleared to avoid processing stale data.

3.3.5 VoiceController Class (Planned for Future Extension)

Class Name: VoiceController

Inherited class: None

Aggregated Classes: Speech recognition engine (e.g., Vosk, Google Speech)

Associated Classes: StateMachine (sends commands)

Data Members:

Type	Name
`Recognizer`	`recognizer`
`Microphone`	`microphone`
`queue.Queue`	`command_queue`
`bool`	`running`

Member Functions:

Return Type	Name
`str`	`listen()`
`str`	`parse_command(audio)`
`void`	`voice_thread()`

Data Structures:

- `command_queue`: a queue for passing recognized commands (e.g., "stop", "go", "search") to the StateMachine.

Processing:

The VoiceController will run a separate thread that continuously listens for voice commands via a microphone. Upon detecting speech, it uses a speech-to-text engine to convert audio to text, matches the text against a set of predefined keywords, and pushes the corresponding command into `command_queue`. The StateMachine will periodically check this queue and alter its behavior accordingly (e.g., pause, resume, or manual override). This module is planned for future integration.

3.4 Navigation and Localization

The system relies on a pre-built static map of the tennis court and uses the Nav2 stack for autonomous navigation to the collection bin. Localization is provided by AMCL (Adaptive Monte Carlo Localization) fused with wheel odometry and IMU data via `robot_localization`.

3.4.1 Offline Mapping

Before autonomous operation, the robot is tele-operated around the court while the SLAM Toolbox builds an occupancy grid. The resulting map (`tennis_field.pgm` and `tennis_field.yaml`) is saved and loaded by Nav2 at runtime. This map serves as the ground truth for global planning and collision avoidance.

3.4.2 Online Localization

At startup, the node publishes an initial pose estimate to the `/initialpose` topic to seed the AMCL particle filter. The initial pose is determined as follows (in order of priority):

1. If the current robot pose can be looked up via TF (`map` → `base_link`), that pose is used.
2. Otherwise, if the user has provided bin coordinates via ROS parameters (`box_x`, `box_y`, `box_yaw_deg`) and enabled `use_initial_pose_as_box`, those coordinates are used as the initial pose (assuming the robot starts at the bin location).
3. If neither is available, the node logs a warning and expects the operator to manually set the initial pose in RViz.

The initial pose is published multiple times to ensure AMCL receives it, reducing convergence time significantly.

3.5 Multi-threading and Multi-processing Architecture

To achieve real-time performance on the Jetson Orin NX while maintaining responsiveness for voice commands, the software combines ROS 2 multi-threaded execution with a dedicated subprocess for speech recognition.

3.5.1 ROS 2 Executor

A `MultiThreadedExecutor` with 8 threads is used to handle callbacks from multiple topics (`/camera/image_color`, `/range_0`) and action clients concurrently. Different callback groups (`MutuallyExclusiveCallbackGroup`, `ReentrantCallbackGroup`) prevent priority inversion between sensor processing and control loops.

3.5.2 Voice Subprocess

The Vosk speech recognizer and the sounddevice audio capture run inside a separate Python process created with `multiprocessing.get_context('spawn')`. This design decision avoids two critical problems:

- The Python Global Interpreter Lock (GIL) would otherwise block audio callbacks during YOLO inference or Nav2 planning, causing audio dropouts and recognition failure.
- Forking a process that already holds CUDA contexts (Jetson GPU) is unsafe; using `'spawn'` creates a clean environment.

Communication between the subprocess and the main process occurs through a `multiprocessing.Queue`. The subprocess pushes `'pause'` or `'resume'` strings, while a lightweight dispatcher thread in the main process invokes the corresponding callbacks.

3.6 Enhanced Search and Alignment Algorithms

3.6.1 Searching with Random Wander

In the `SEARCHING` state, if no tennis ball is detected for five consecutive frames, the robot rotates by 15° increments. After completing a full 360° rotation without any detection, it performs a *random wander*: a random yaw change (±60°) followed by a short forward movement (0.3–0.6m). This breaks out of local minima where the ball might be just outside the camera's field of view.

3.6.2 Oscillation Damping in ROTATING

The proportional controller that aligns the ball to the image centre uses an adaptive damping factor to prevent overshoot and oscillation. When the sign of the error (`offset`) flips between consecutive frames, the damping factor is multiplied by 0.6. The commanded rotation angle is:

$$\text{angle} = \frac{\text{offset}}{\text{IMG\_W}/2} \times 50^\circ \times \text{damping}$$

with a hard clamp of ±15°. This mechanism ensures smooth convergence even when the ball is near the image centre.

3.6.3 Visual Servoing During Grabbing

While the robot moves forward towards the ball (linear speed 0.18m/s), the control thread continuously reads the latest detection results and publishes a heading correction:

$$\omega = -k \cdot \text{offset}, \quad k = 0.3/320$$

This keeps the ball centred despite chassis drift. The ToF sensor runs at up to 100Hz and provides the primary stop condition (distance 0.13m). If the ToF reading is unavailable, the robot relies solely on visual feedback, but the gripper will not close until the distance threshold is met.

3.7 System Parameters and Initialization

All key parameters are exposed as ROS 2 parameters or constants defined in the source code. Table 6 lists the most important ones.

Parameter	Value	Description
`TOF_TRIGGER_M`	0.13m	Distance to close gripper
`DETECT_CONFIRM_FRAMES`	3	Consecutive detections needed to enter ROTATING
`LOST_TRIGGER_FRAMES`	5	Frames without detection before search action
`ROTATE_TOLERANCE_PX`	30px	Allowable offset from image centre
`CHASSIS_LINEAR_SPEED`	0.3m/s	Max forward speed during grabbing
`NAV2_TIMEOUT_SEC`	90.0s	Max time allowed for Nav2 navigation
`NAV2_MAX_RETRY`	2	Number of retries after Nav2 failure
`VOICE_GAIN`	8.0	Software gain for microphone input

Table 6: Key system parameters

The bin position in the map frame can be specified at launch using the parameters `box_x`, `box_y`, `box_yaw_deg`. Alternatively, setting `use_initial_pose_as_box` to `true` records the robot's starting pose as the bin location. This flexibility allows the system to adapt to different court layouts without recompilation.

3.8 Error Handling and Recovery

The system incorporates several robustness mechanisms:

- **Grabbing timeout:** If the ToF distance does not drop below 0.13m within 10seconds, the robot aborts the attempt and goes to `HOMING` (releasing any partially grasped ball).

- **Nav2 retry:** If a navigation goal is aborted or cancelled, the node automatically retries up to `NAV2_MAX_RETRY` times. After each failure, the robot waits for the pause flag to clear (if a voice pause caused the cancellation) and then re-sends the goal.
- **Voice pause cancellation:** When the user says “”, the robot immediately stops all motion (via zero velocity commands) and cancels the active Nav2 goal. After “”, the state machine resumes and will re-enter the navigation routine from the current position.
- **Recovery from detection loss:** If the ball is lost during `ROTATING` or `GRABBING`, the state machine falls back to `SEARCHING` and the search process restarts.

3.9 Software Dependencies and Execution Environment

The system is built on **ROS 2 Humble** running on Ubuntu 22.04 (ARM64) on the Jetson Orin NX. Key dependencies include:

- `robomaster_ros` – driver for DJI RoboMaster EP (actions for chassis, arm, gripper, ToF).
- `nav2_bringup`, `nav2_msgs` – navigation stack with DWB local planner.
- `slam_toolbox` – offline mapping.
- `robot_localization` – EKF fusion of odometry and IMU.
- `ydlidar_ros2_driver` – YDLidar X3 Pro interface.
- `cv_bridge`, `OpenCV` – image processing and HUD rendering.
- `PyTorch` (CUDA 11.4+) – YOLOv5 inference on GPU.
- `vosk` and `sounddevice` – offline Chinese speech recognition.

The system is launched using a custom ROS 2 launch file that starts all required nodes (Nav2, SLAM, robot driver, YDLidar, EKF) before invoking the main `tennis_pick_nav2` node.

4 User Interface Design

This section describes the user interface design.

The system architecture is shown in the diagram and adopts a modular design, mainly divided into two major data processing pipelines: the video stream pipeline and the distance data pipeline.

The video processing pipeline acquires the raw video stream from the camera. After processing by the video capture module, it is transmitted to the video window UI for display. Users can manipulate the interface to zoom in/out or switch to full screen, thereby obtaining a better viewing experience.

The distance data processing pipeline collects raw measurement data through the distance sensor. After the data acquisition module processes it, on one hand, it is transmitted to the log window UI for real-time display to facilitate on-site monitoring and record review; on the other hand, the data is sent to a parsing/formatting module for processing, converting it into a structured format that can be used for subsequent playback and system debugging.

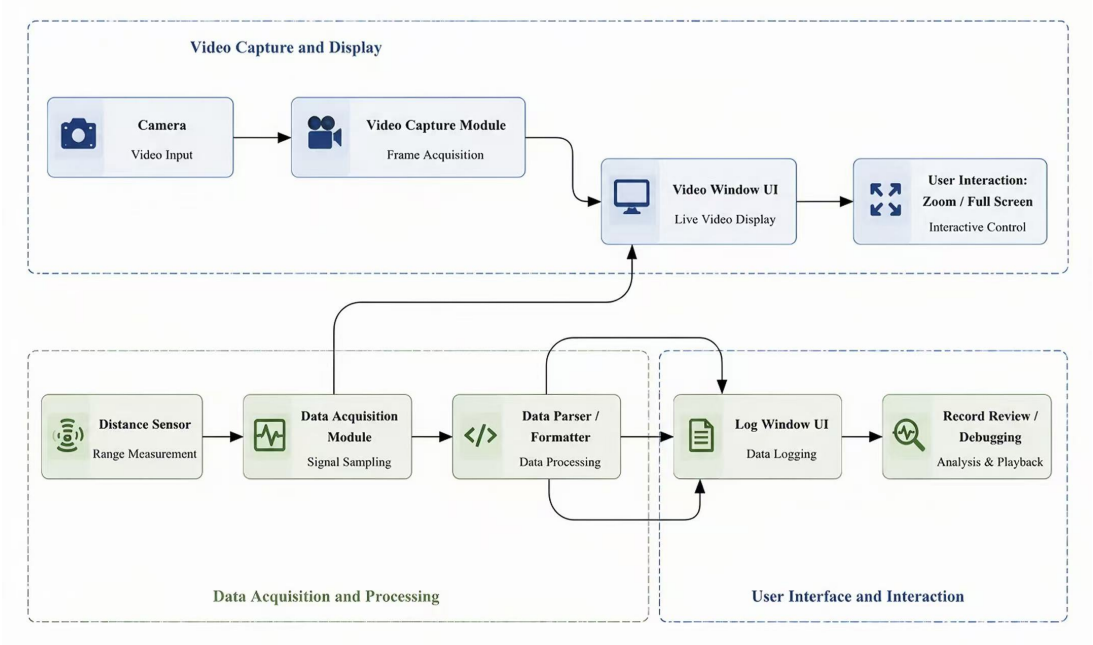

Figure 4: Log Window

The entire architecture clearly separates the acquisition, processing, and display of video and distance data, ensuring real-time performance while providing flexible support for data analysis and system maintenance.

4.1 Operator Console Interface

To provide intuitive monitoring and control of the autonomous tennis ball retrieval system, a graphical user interface (GUI) is developed. The console aggregates live camera feed, robot status, mission controls, and system logs into a single window, enabling both autonomous operation supervision and manual override when necessary.

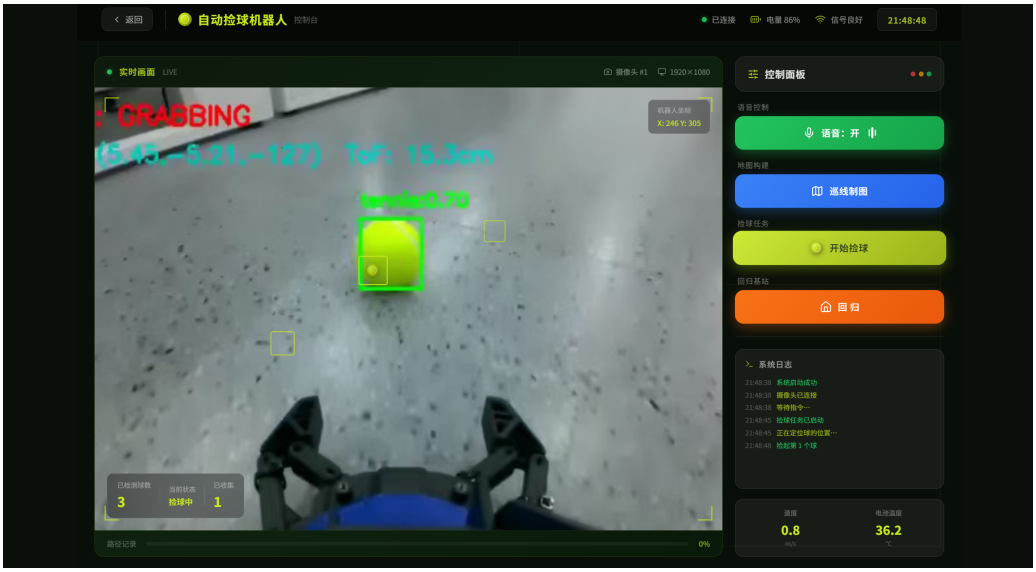

Figure 5: Operator console interface of the tennis ball retrieval system

The interface is divided into several functional panels, as labelled in Figure 5.

4.1.1 Real-time Video Panel

The left portion of the console displays the live RGB stream from the onboard camera (1920×1080 resolution). YOLOv5 detection bounding boxes, confidence scores, and the current finite state machine state are overlaid on the video. This panel helps operators verify that the robot is correctly perceiving tennis balls and obstacles.

4.1.2 Control Panel

The centre-right area contains all mission controls and status indicators:

- **Robot coordinates:** Displays the robot's current position (X, Y) in the SLAM map frame, updated at 5Hz.
- **Voice control toggle:** Enables or disables voice command recognition ("start" indicates voice is active). A dropdown or button allows the operator to manually trigger pause/resume.
- **Map building:** Launches the SLAM Toolbox offline mapping routine; the robot can be teleoperated to scan the court.
- **Line patrol mapping:** Alternative semi-autonomous mode for boundary following (reserved for future use).
- **Ball collection task:** Starts the full autonomous state machine (SEARCHING $\rightarrow \ldots \rightarrow$ HOMING). Once pressed, the robot begins hunting for tennis balls.
- **Return to base / docking:** Commands the robot to navigate back to the collection bin (or starting position) using Nav2, even if the current mission is paused.
- **Collected ball counter:** Displays the number of balls successfully grasped and placed during the current session.
- **Current state:** Shows the active FSM state (e.g., "standby" , "Searching", "garbbing", "placing").
- **Battery temperature:** Monitored via the RoboMaster EP's battery management system; displayed in degrees Celsius.
- **Path recording:** A log of waypoints or travelled path (optional, for debugging).

4.1.3 System Log Panel

The bottom section of the console maintains a scrollable log of system events, warnings, and status changes. Each entry is timestamped. Typical messages include:

- System startup and camera connection confirmation.
- Mission start/stop commands.
- Nav2 goal status (accepted, succeeded, aborted).
- Voice command recognition results ("stop", "start").
- Grasping success or timeout events.

The log panel assists operators in diagnosing unexpected behaviours without accessing the terminal.

4.1.4 Connection and Power Status

The top bar indicates the robot's connection state ("connect"), battery level (86

All UI components are implemented using `PyQt5` or a web-based dashboard (Flask + WebSocket) that subscribes to the same ROS 2 topics as the main node. The console runs on a remote laptop connected to the robot's Wi-Fi network, allowing an operator to monitor multiple robots or intervene from a safe distance.

4.2 Remote Monitoring with RViz

In addition to the operator console, the system leverages **RViz2** as a high-fidelity visualisation tool for debugging and supervision. RViz runs on a remote laptop connected to the same ROS 2 network and subscribes to the following topics:

- `/map` – occupancy grid built by SLAM Toolbox, displayed as a costmap overlay.
- `/amcl/particlecloud` – AMCL particle cloud showing the robot's localisation confidence.
- `/tf` – robot pose (base_link, odom, map frames) visualised as a 3D model.
- `/plan` – global plan published by Nav2 (A* path).
- `/cmd_vel` – velocity commands visualised as arrows.
- `/camera/image_color` – optional camera feed overlaid on the map.

Figure 6 shows a typical RViz layout used during field tests. The operator can manually set initial poses, inspect costmap updates, and monitor Nav2's local planning behaviour. RViz also provides buttons to send Nav2 goals interactively, which is useful for overriding the autonomous mission.

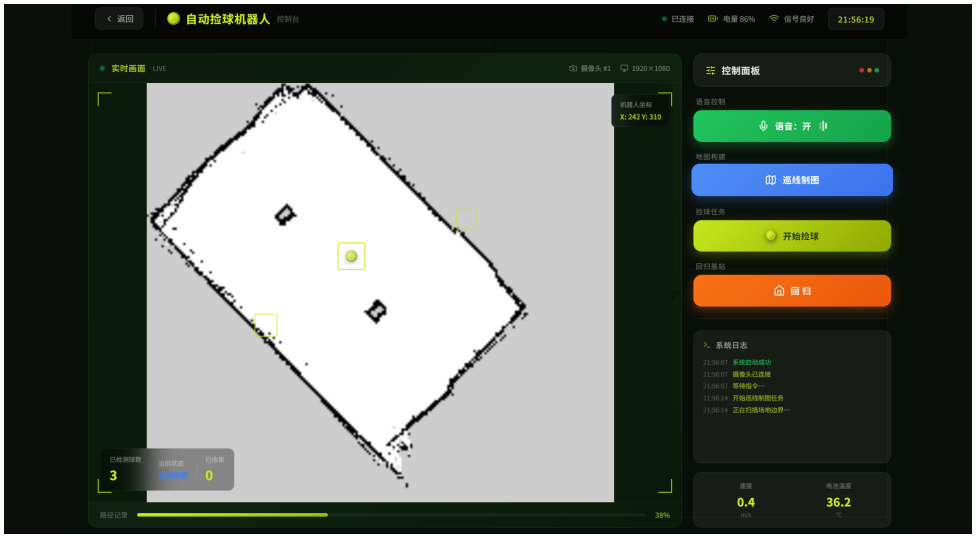

Figure 6: RViz2 visualisation of SLAM map, AMCL particles, and planned path

4.3 Voice Feedback and Status Indicators

The operator console and RViz both display real-time voice command feedback. When the Vosk subprocess recognises a keyword, the following visual cues appear:

- A pop-up notification in the console: `\stop"` or `\start"`.
- The current state badge changes colour (red for paused, green for active).
- A log entry is appended to the system log panel with a timestamp.

This feedback loop assures the operator that voice commands have been received and acted upon, which is critical when the robot is operating far from the laptop.

4.4 Advanced Configuration Panel

For advanced users or researchers, the console includes a collapsible configuration panel that allows run-time modification of key parameters without recompiling. Parameters are stored in a YAML file and can be updated via ROS 2 dynamic reconfigure or a simple GUI input. Table 7 lists the adjustable parameters.

Parameter	**Default**	**Description**
Detection confidence threshold	0.70	Minimum confidence for ball detection
ToF trigger distance (m)	0.13	Distance to close gripper
Rotation tolerance (px)	30	Allowed centre offset for alignment
Chassis linear speed (m/s)	0.18	Forward speed during grabbing
Nav2 timeout (s)	90.0	Maximum navigation time per goal
Voice gain	8.0	Software amplification for microphone

Table 7: User-configurable parameters in the UI

The panel also provides buttons to save the current configuration and to restore factory defaults.

References

[1] dji. *dji service and support*. 2024. URL: https://www.dji.com/cn/support/product/robomaster-ep.

[2] dji. *dji Technical parameters of the education platform*. 2024. URL: https://www.dji.com/cn/edu-hub/specs.

[3] dji. *RoboMaster EP technical specification*. 2023. URL: https://www.dji.com/cn/robomaster-ep/specs.

[4] dji. *RoboMaster EP User Manual*. 2021. URL: https://www.yingjianzhijia.com/sms/28657.html.

[5] Gitee. *George/ICRA-RM-Sim2Real-Baseline*. 2024. URL: https://gitee.com/tb5zhh/icra-rm-sim2real-baseline/blob/master/README.md.

[6] Ultralytics. *YOLOv5: Real-Time Object Detection Library*. 2021. URL: https://github.com/ultralytics/yolov5.